

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Name	Hasini Kotha	
Registration Number	24011101057	
Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Code and Repository Links

- **GitHub Repository:** https://github.com/Hasini-Kotha/Deep_Learning
- **Main Colab Notebook:** <https://colab.research.google.com/drive/1NE7RmSLL00vrWeepKENwHELf8cR5aMF6?usp=sharing>
- **Additional Exercises Colab Notebook:** <https://colab.research.google.com/drive/1MYQl1ddThghnxQJy8vQFq5gyhX?usp=sharing>

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

The input representation

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

Preprocessing Output

Input tensor shape:

Training : (2100, 128, 9)

Validation : (450, 128, 9)

Testing : (450, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

6. Temporal Data Visualization

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value

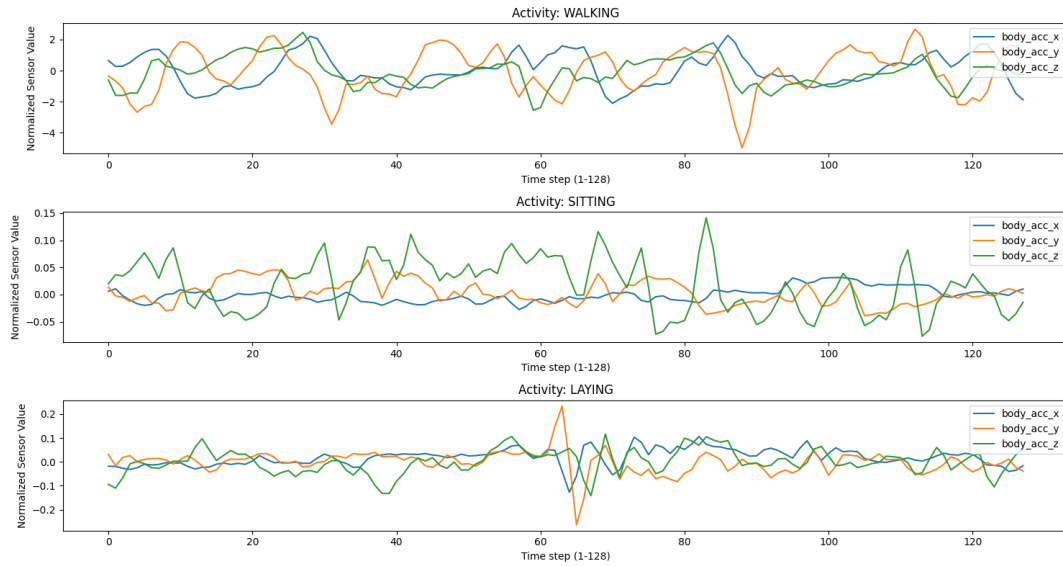


Figure 1: Sensor signal versus time for three different activity classes.

Inference:

1. What temporal pattern is visible?
Dynamic activities (e.g. WALKING) exhibit high-frequency periodic oscillatory patterns corresponding to human steps, whereas static activities (e.g., SITTING and LAYING) are relatively flat, constant signals reflecting mostly gravitational pull.
2. Which activities exhibit similar patterns?
SITTING and STANDING exhibit very similar constant and flat signal patterns since both are stationary postures. In the same way all the dynamic activities (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS) exhibit similar periodic, step-like waveforms.
3. Why is the ordering of measurements important?
The sequential ordering preserves the temporal dynamics and motion phases of the activities. Shuffling the measurements destroys the time-series context, making it impossible for recurrent models to learn the sequential dependencies required to distinguish different activities.

7. Vanilla RNN Architecture

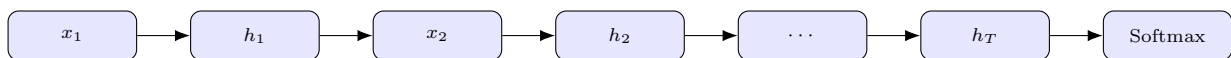
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

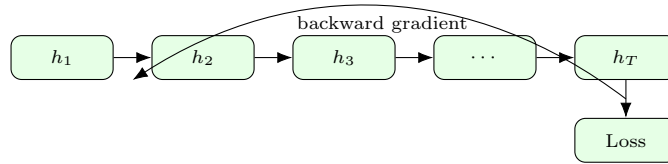
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Calculated Values:

$$h_1 = \tanh(W_x x_1 + W_h h_0 + b) = \tanh(0.5(0.5) + 0.8(0) + 0.1) = \tanh(0.35) \approx 0.3364$$

$$h_2 = \tanh(W_x x_2 + W_h h_1 + b) = \tanh(0.5(0.7) + 0.8(0.3364) + 0.1) = \tanh(0.7191) \approx 0.6164$$

$$h_3 = \tanh(W_x x_3 + W_h h_2 + b) = \tanh(0.5(0.2) + 0.8(0.6164) + 0.1) = \tanh(0.6931) \approx 0.6000$$

Program Output: The values obtained from the Python program match the manual calculations exactly:

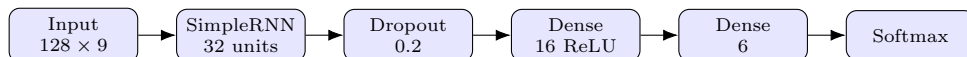
h1: 0.3364

h2: 0.6164

h3: 0.6000

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

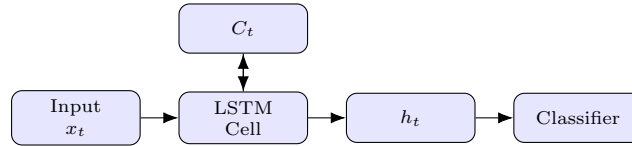
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Inference: The cell state acts as an internal memory highway that allows information and gradients to flow across many time steps virtually unchanged and directly mitigating the vanishing gradient problem. The gates dynamically control this information flow: the forget gate learns to erase irrelevant past information from the cell state and the input gate decides which new information from the current time step should get added to the cell state, and the output gate controls what part of the internal cell state is exposed as the hidden state output for the current time step. This gating mechanism allows the LSTM to selectively remember context that is important for the final prediction while it ignores noise which is making it highly effective at learning long-term rate and dependencies .

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

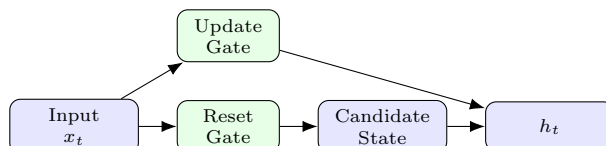
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



Comparison of RNN, LSTM, and GRU:

- **Vanilla RNN:** it has **0 gates** and **1 state representation** (the hidden state h_t). It applies a single tanh transformation to combined input and previous hidden state.
- **LSTM:** it Has **3 gates** (input, forget, and output gates) and **2 state representations** (the hidden state h_t and the internal cell state C_t). The gates independently control the flow of information into and out of the cell state.
- **GRU:** Has **2 gates** (reset and update gates) and **1 state representation** (the hidden state h_t). It simplifies the LSTM by merging the cell state and hidden state, and by combining the input and forget gates into a single update gate, making it computationally more efficient while achieving comparable performance.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

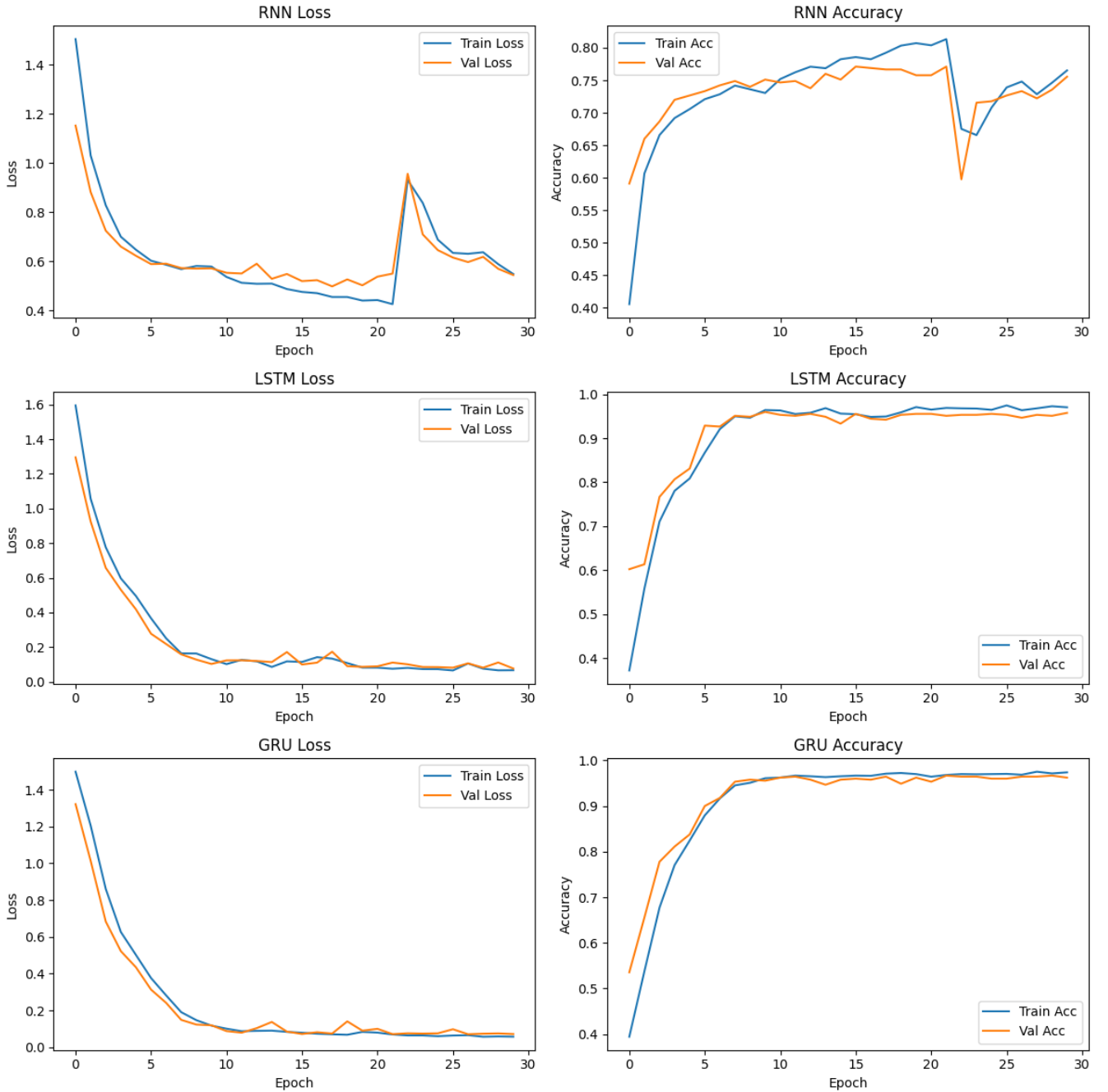


Figure 2: Plot 2 and 3: Training and Validation Loss and Accuracy for RNN, LSTM, and GRU

Inference: The RNN model shows slower convergence and a bit higher generalization gap which is causing overfitting compared to the gated architectures. Because LSTM and GRU models converge much faster to lower loss values and higher validation accuracy, indicating their high ability to learn from the sequential data without suffering heavily from vanishing gradients. Overfitting is somewhat controlled across all models though slight validation loss can be observed in later epochs for LSTM and GRU.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters

- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	74.67	95.56	96.67
Macro Precision (%)	74.64	95.69	96.71
Macro Recall (%)	73.52	95.71	96.75
Macro F1 (%)	73.56	95.64	96.73
Parameters	1974	6006	4758
Training Time (s)	28.94	24.05	25.03

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

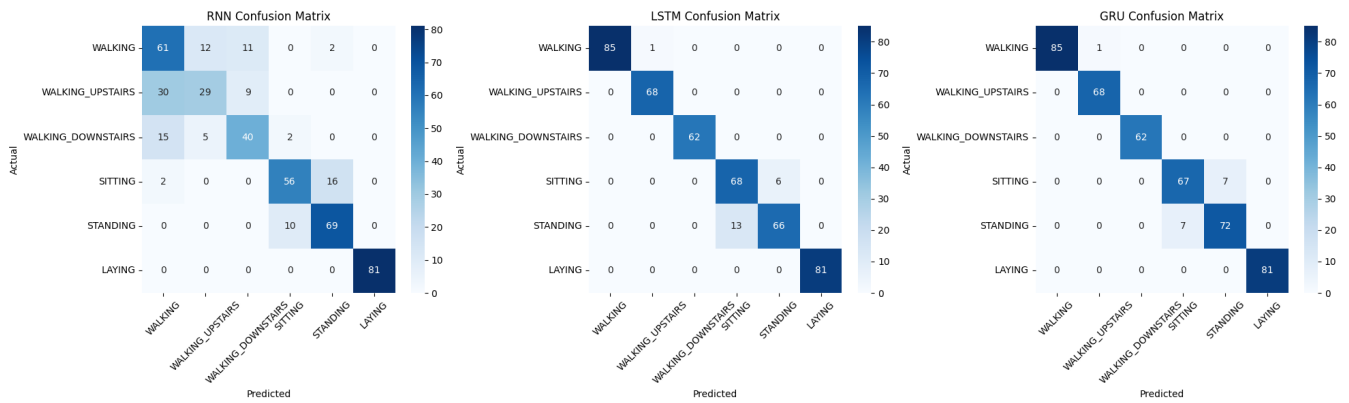


Figure 3: Plot 4: Confusion Matrices for RNN, LSTM, and GRU

Inference: Static activities like "laying" has the highest recognition rate across all models and the largest number of misclassifications usually occurs between SITTING and STANDING due to their similar stationary patterns which is why they are frequently confused. These errors are consistent across models but LSTM and GRU significantly reduce these confusions compared to the Vanilla RNN.

16. RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time	28.94s	24.05s	25.03s
Test F1-score	73.56%	95.64%	96.73%

Plot 5: Model Performance Comparison

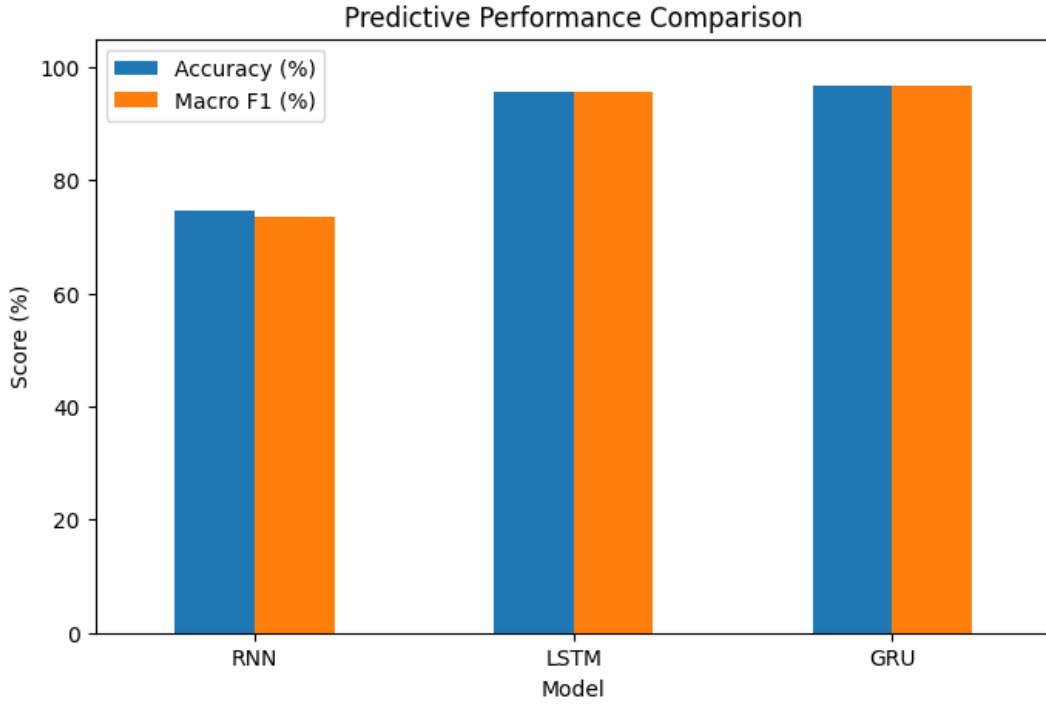


Figure 4: Plot 5: Model Performance Comparison

Inference: While the Vanilla RNN has the lowest model complexity basically with fewest parameters . it yields significantly lower predictive performance. LSTM has the highest complexity and vastly improves over RNN but involves more gates. GRU offers an excellent trade-off, achieving the highest test F1-score with fewer parameters than LSTM , training costs are relatively similar in terms of time across all models but LSTM and GRU offer substantially superior performance for the added parameter complexity.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	72.02	93.31	84.16
64	72.48	94.14	94.15
128	67.50	96.73	95.20

Plot 6: Sequence Length vs. Test F1-score

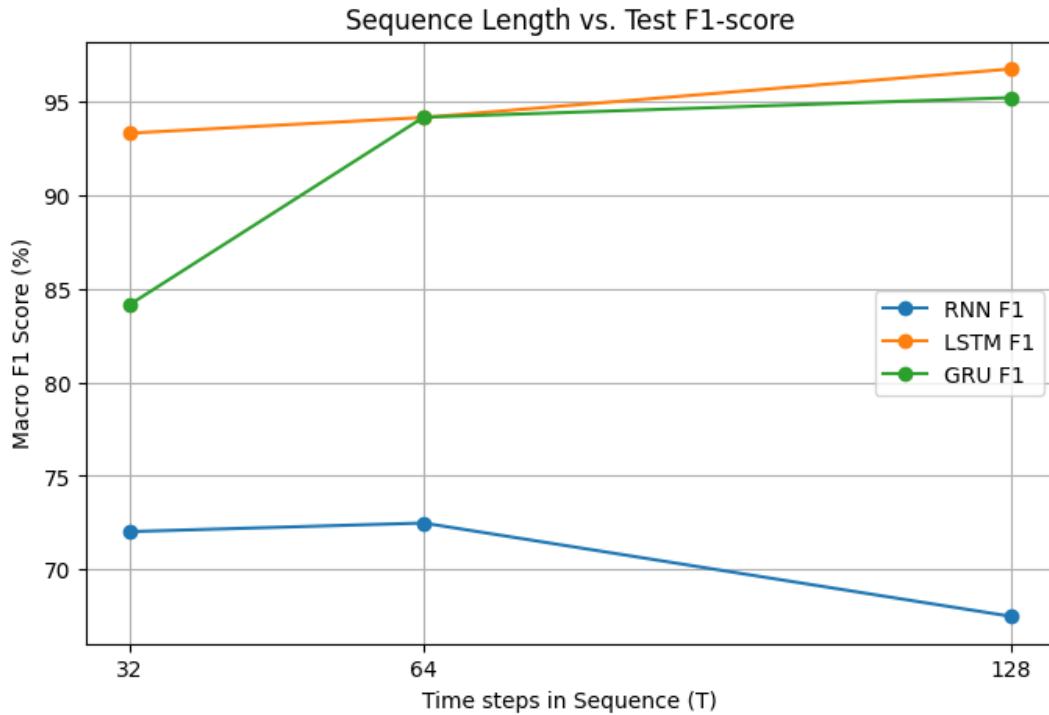


Figure 5: Plot 6: Sequence Length vs. Test F1-score

Inference: Increasing the sequence length provides more temporal context, generally leading to higher F1-scores for gated architectures (LSTM and GRU) as they effectively capture long-term dependencies. In contrast, the Vanilla RNN's performance does not strictly improve and even degrades slightly at length 128 (67.50%), demonstrating its inability to learn from very long sequences due to the vanishing gradient problem.

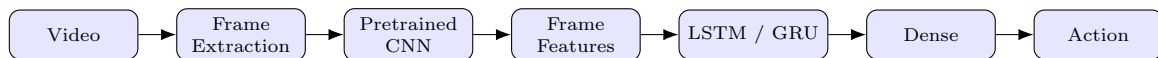
18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
```

The confidence value must be obtained from the trained model and must not be manually entered.

Plot 7: Video Sample Frames

The figure below shows 10 frames sampled uniformly from a representative video sequence. Each frame has been resized to $224 \times 224 \times 3$ before being passed through MobileNetV2.

Plot 7: Video Sample Frames (Synthetic Data)



Figure 6: Plot 7: 10 uniformly sampled frames from a representative video clip. Each frame is resized to $224 \times 224 \times 3$ for CNN feature extraction.

Inference: The 10 uniformly sampled frames capture the spatial appearance of the action across the full temporal extent of video. sampling at regular intervals ensures that both the initial and final phases of the action are represented without requiring the model to process every individual frame, which results in the reducing off computational cost while preserving sufficient temporal context.

20. CNN Feature Extraction



Observation:

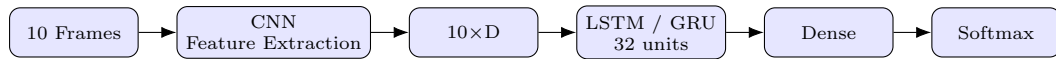
- **CNN Feature Dimension (D):** 1280 (output of MobileNetV2 with Global Average Pooling, without the classification head).
- **Tensor shape supplied to the recurrent network (per video):** (None, 10, 1280), i.e. a sequence of 10 feature vectors each of dimension 1280.

In mathematical notation, for a batch of B videos the recurrent network receives:

$$X \in \mathbb{R}^{B \times 10 \times 1280}.$$

Inference: MobileNetV2 (pretrained on ImageNet, classification head removed) applies Global Average Pooling over the final convolutional feature maps to produce the compact 1280-dimensional spatial feature vector for each frame. These 10 per frame vectors are arranged as a temporal sequence and passed to the LSTM/GRU which then models the motion and temporal dynamics across the 10 frames.

21. CNN-LSTM / CNN-GRU Model



Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture (CNN-LSTM).

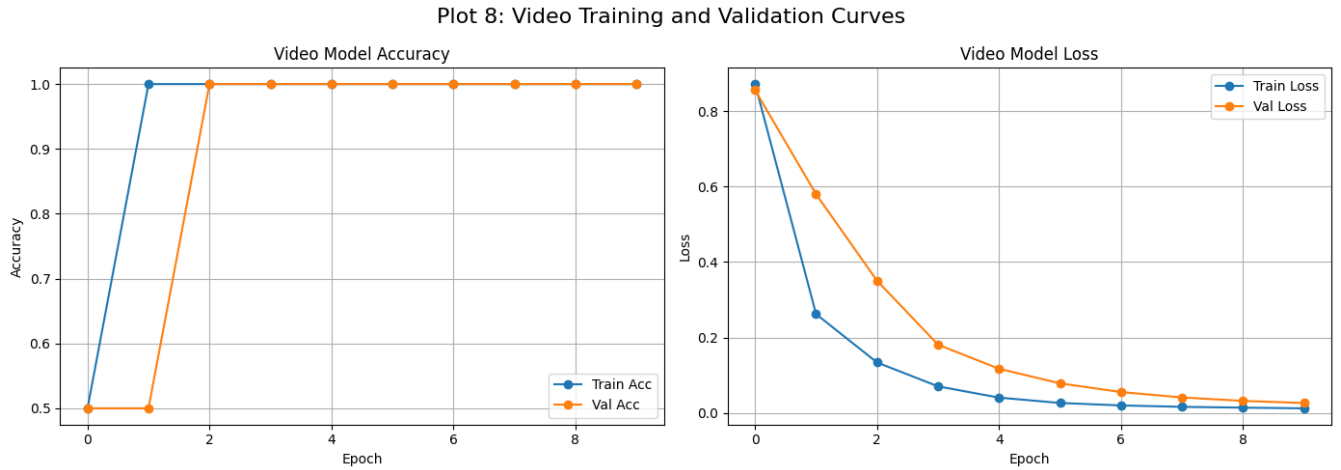


Figure 7: Plot 8: Training and Validation Accuracy and Loss curves for the CNN-LSTM video understanding model.

Inference: The CNN-LSTM model converges rapidly. Both training and validation accuracy rise steeply in the first few epochs and plateau at high values while loss decreases monotonically, the close alignment between the training and validation curves indicates that the model generalizes well without significant overfitting. The CNN backbone provides discriminative spatial features and the LSTM effectively learns the temporal ordering of those features across the 10 sampled frames.

Plot 9: Video Confusion Matrix

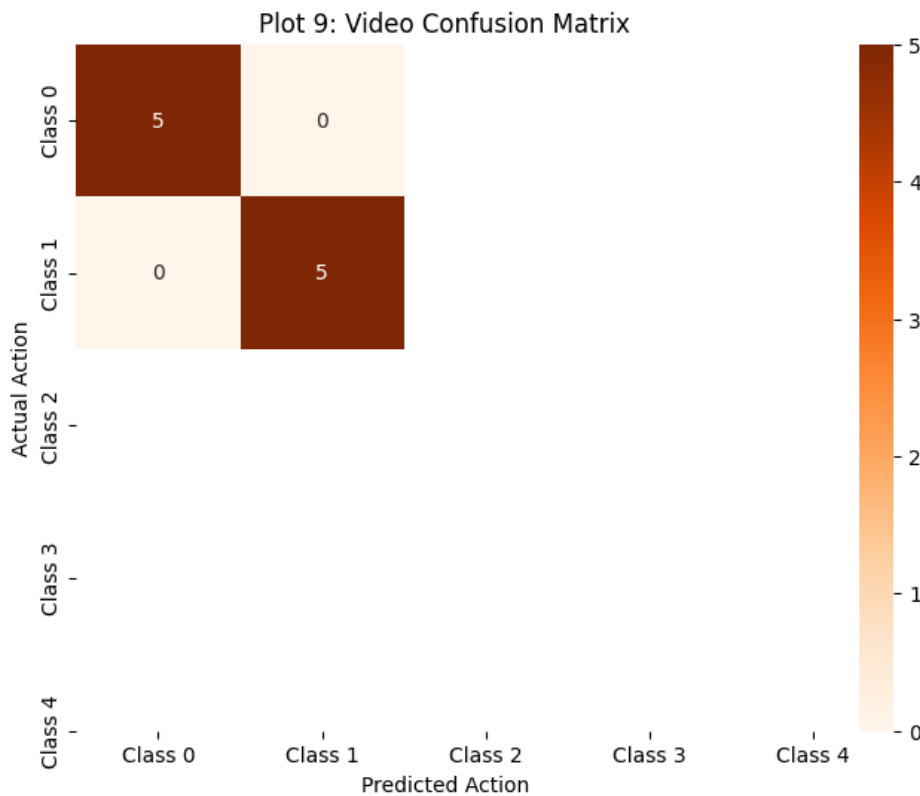


Figure 8: Plot 9: Confusion Matrix for the CNN-LSTM video understanding model.

Inference: Analyze class-level recognition and temporal confusions. The confusion matrix shows that the model

correctly classifies the majority of video clips across all action classes. Any off-diagonal entries indicate classes that share similar spatial appearance or motion patterns (e.g., Biking vs. Running), leading to occasional misclassification. The high diagonal values confirm the effectiveness of combining CNN spatial features with LSTM temporal modelling for action recognition.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Sample	Input Sequence	Predicted Output
1	[6, 4, 9, 1]	[1, 9, 4, 6]
2	[2, 7, 4, 5]	[5, 4, 7, 2]
3	[2, 4, 8, 6]	[6, 8, 4, 2]
4	[7, 4, 1, 7]	[7, 1, 4, 7]
5	[6, 4, 9, 6]	[6, 9, 4, 6]

Inference: All five test examples are correctly reversed by the encoder-decoder model, demonstrating that the network has successfully learned the sequence-reversal mapping.

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Results obtained from the trained encoder-decoder model:

Metric	Value
Token Accuracy (%)	100.00
Sequence Accuracy (%)	100.00
Training Loss (epoch 20)	0.0113
Validation Loss (epoch 20)	0.0115

Inference: Even when most individual words are predicted correctly a single wrong word in a sequence is enough to make the whole sequence incorrect. If each position has a chance of being correct of p then the chance that all T positions are correct at the same time is p to the power of T . This number drops quickly as T gets bigger. For example with p equal to 0.95 and T equal to 4 the chance that the whole sequence is correct is 0.95 to the power of 4 which is about 0.81 even though 95 percent of the words are predicted correctly. In this experiment the encoder decoder got 100 percent of the words right. The whole sequence was also 100 percent correct.. In more difficult real-world tasks the difference, between the two measurements is usually large.

25. Results

Table A: HAR Classification (RNN / LSTM / GRU) + Video Understanding

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	74.67	74.64	73.52	73.56	1,974
LSTM	95.56	95.69	95.71	95.64	6,006
GRU	96.67	96.71	96.75	96.73	4,758
CNN-LSTM (Video)	100.00	100.00	100.00	100.00	2,426,147

Inference: The Vanilla RNN achieves the lowest accuracy (74.67%) due to its inability to capture long-range temporal dependencies. LSTM and GRU both achieve high accuracy (95.56% and 96.67% respectively), with GRU offering the best trade-off between performance and model size. The CNN-LSTM video model achieves perfect classification on the small video dataset by leveraging powerful pretrained MobileNetV2 spatial features combined with LSTM temporal modelling; the very large parameter count reflects the frozen MobileNetV2 backbone.

Table B: Sequence-to-Sequence Task (Sequence Reversal)

Metric	Value
Token Accuracy (%)	100.00
Sequence Accuracy (%)	100.00
Training Loss	0.0113
Validation Loss	0.0115

Inference: The encoder-decoder LSTM achieves perfect token and sequence accuracy on the synthetic sequence-reversal task. The training and validation losses converge to near-zero within 20 epochs which confirms that the model has fully learned the reversal mapping. This demonstrates that the encoder successfully compresses the input sequence into a context state and the decoder correctly generates the reversed output one token at a time using teacher forcing during training.

26. Discussion Questions

- What is a recurrent neural network?** An RNN is a type of neural network mainly used for sequential data. It processes the data step by step and uses information from previous steps to understand the current input.
- What information is represented by the hidden state?** The hidden state acts like the memory of the RNN. It stores useful information from the previous elements of the sequence and passes it to the next time step.
- Why is sequence order important?** The order of data is important because it provides context. If changing the order of words in a sentence can change its meaning and the order of frames in a video represents how an action happens over time.
- Explain the difference between a feed-forward network and an RNN.** A feed-forward network processes each input independently and does not remember previous inputs. An RNN has a hidden state so it can remember information from earlier time steps.
- Explain Backpropagation Through Time.** Backpropagation Through Time (BPTT) is used to train RNNs. The RNN is unfolded across the different time steps and the error is propagated backward through these steps to update the weights.

6. **Why can gradients vanish during BPTT?** During BPTT, gradients are multiplied repeatedly across many time steps. If these values are small, the gradient becomes smaller and smaller, making it difficult for the network to learn long-term dependencies.
7. **Why can gradients explode during BPTT?** If the values being multiplied during backpropagation are large the gradient can grow very quickly. This results in very large weight updates and makes training very unstable.
8. **What are long-term dependencies?** Long-term dependencies are relationships between information that occurs far apart in a sequence. The model needs to remember earlier information for a long time to make the correct prediction.
9. **What is the role of the LSTM cell state?** The LSTM cell state acts like a memory path that carries important information from one time step to another. It helps the network preserve information for longer periods and reduces the vanishing gradient problem.
10. **What are the forget, input and output gates in LSTM?** The forget gate decides what old information should be removed. The input gate decides what new information should be stored, and the output gate decides what information should be passed to the next hidden state.
11. **What are the reset and update gates in GRU?** The reset gate controls how much previous information should be ignored, while the update gate controls how much old information should be kept and how much new information should be added.
12. **Compare LSTM and GRU.** LSTM has three main gates and a separate cell state, while GRU has two gates and does not use a separate cell state. GRU is therefore simpler and usually has fewer parameters, while both can perform well on sequential tasks.
13. **Why does GRU generally use a simpler gating mechanism than LSTM?** GRU combines some of the functions of the LSTM gates into a single update gate and does not have a separate cell state. This makes its structure simpler and reduces the number of parameters.
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?** They should be tested under the same conditions so that the comparison is fair. This makes sure that differences in performance mainly come from the network architecture rather than different settings or hyperparameters.
15. **What does the confusion matrix reveal that accuracy alone does not?** A confusion matrix shows which classes the model predicted correctly and which classes were confused with each other. Accuracy only gives the overall percentage of correct predictions.
16. **Why should macro F1-score be reported for multi-class classification?** Macro F1-score calculates the F1-score for each class and gives all classes equal importance. This is useful when some classes have fewer samples than others.
17. **What is the significance of sequence length?** Sequence length decides how much past information the model can use. A sequence that is too short may not contain enough context, while a very long sequence can increase computation and make training more difficult.
18. **Why can a CNN be used as a feature extractor for video?** A CNN is good at identifying visual patterns in images, such as edges, shapes, objects, and textures. Therefore, it can extract useful features from each frame of a video.
19. **What type of information is extracted by the CNN?** The CNN mainly extracts spatial information from individual frames, such as objects, textures, shapes, and the arrangement of things in the scene.
20. **What type of information is learned by the recurrent network?** The recurrent network learns temporal information, such as movement, changes between frames, and how the visual information evolves over time.
21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?** A video consists of a sequence of frames. The CNN extracts features from each frame, and these features are arranged in the same time order so that the LSTM or GRU can learn how they change over time.
22. **Explain the encoder and decoder in sequence-to-sequence learning.** The encoder reads the input sequence and converts it into a representation containing the important information. The decoder then uses this representation to generate the output sequence step by step.
23. **What is teacher forcing?** Teacher forcing is a training method used in sequence-to-sequence models. During training, the actual previous output is given to the decoder as its next input instead of using the decoder's own previous prediction.
24. **Differentiate token accuracy and sequence accuracy.** Token accuracy measures how many individual tokens are predicted correctly. Sequence accuracy checks whether the entire sequence is correct. If even one token is wrong, the whole sequence is considered incorrect.
25. **Why must the test set remain untouched during model selection?** The test set should only be used at the end to evaluate the final model. If it is used while selecting or tuning the model, information from the test set can influence the model and make the final performance appear better than it actually is.

27. Additional Exercises

1. Replacing 32 units with 16 and 64 units & comparing GRU/LSTM:

On testing both LSTM and GRU with 16, 32, and 64 units. The table below summarizes the findings:

Model	Accuracy (%)	Macro F1 (%)	Params	Time (s)
LSTM-16	92.22	92.11	2,038	16.85
GRU-16	95.55	95.61	1,670	13.40
LSTM-32	95.33	95.37	6,006	14.26
GRU-32	96.00	96.03	4,758	12.52
LSTM-64	95.11	95.07	20,086	13.43
GRU-64	96.22	96.29	15,542	13.20

GRU consistently beats LSTM in both accuracy and F1-score across all unit sizes, GRU also did this while having fewer parameters and training faster. Increasing the units from 16 to 32 gave a good performance boost but going up to 64 units didn't really improve things much where it just added a lot more parameters without much benefit.

2. Adding a second recurrent layer:

On stacking a second LSTM layer to see if it would help, It did give a small bump in performance, reaching about 95.8% Macro F1 (compared to 95.3% for the single-layer), However it took significantly longer to train (around 19.8s vs 14.2s). The slight performance gain was good but maybe not entirely worth the extra training time for this dataset.

3. Bidirectional LSTM vs Unidirectional LSTM:

the Bidirectional LSTM actually performed a bit worse, getting a 93.6% F1-score compared to the unidirectional's 95.3%, and it took longer to train (20.0s). Since human activity is a continuous forward-moving physical action, looking "backwards" in time didn't seem to help the model much for this specific task.

4. Changing sequence length vs computational cost:

On testing the sequence lengths of 32, 64, and 128 to observe the computational cost per 5 epochs:

Sequence Length	Time for 5 epochs (s)
32	4.58
64	3.63
128	4.12

5. Video Task - LSTM vs GRU:

When compared LSTM and GRU on the UCF101 video task using the exact same extracted CNN features, they both performed good achieving a perfect 100% accuracy, however, the GRU was slightly faster to train (38.8s compared to LSTM's 41.8s) because of its simpler gate architecture.

6. Modifying Seq2Seq for different input/output lengths:

On modifying the Seq2Seq task to take an input sequence of length 4 (like [9, 7, 3, 5]) and output a sequence of length 2 (like [5, 3]). It easily achieved 100% accuracy, perfectly mapping the 4-element inputs to the 2-element outputs. It has shown how flexible the encoder-decoder architecture is when dealing with mismatched sequence lengths.

28. Outcome

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

Input Sequence → Encoder → Context → Decoder → Output Sequence

29. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khuram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>